

실습 환경과 API 키 설정

파이썬 · 코랩 · 깃허브 · API의 정의와 상호 관계

비정형 사료의 디지털화 · 4차시 보강

용어 정의

이후 설명은 다음 여섯 개념을 전제로 합니다

	정의	3차시에서의 대응 요소
1 깃허브	코드와 자료를 공개해 두는 원격 저장소	실습 노트북과 자료의 배포처
2 파이썬	프로그래밍 언어. 기술한 명령을 인터프리터가 실행한다	코드 칸에 기술된 명령
3 구글 코랩	브라우저에서 파이썬을 실행하는 클라우드 서비스	실행 버튼 · 구글 서버에서 연산
4 API	프로그램이 다른 프로그램의 기능을 호출하는 규격	⑥칸 document_text_detection()
5 API 키	호출자를 식별하고 사용량을 집계하는 자격증명	비전 .json · 제미나이 문자열
6 파이프라인	앞 단계의 출력이 다음 단계의 입력이 되는 처리 구조	④ → ⑥ → ⑨ 칸의 연결

깃허브 — 원격 코드 저장소

수업 자료를 이곳에 올린 이유

1 단일 주소

메일 첨부로 배포하면 참가자별로 상이한 사본이 생성됩니다. 저장소는 주소가 하나입니다.

2 수정의 즉시 반영

3차시 종료 후 오류를 수정했습니다. 이후 접속하는 참가자는 수정본을 엽니다. 재배포 절차가 필요하지 않습니다.

3 주소의 지속성

URL이 변경되지 않으므로 워크숍 종료 후에도 동일한 경로로 접근할 수 있습니다.

4 변경 이력 관리

본래 다수의 작업자가 코드를 공동 수정하기 위한 도구입니다. 이 워크숍에서는 배포 기능만 사용합니다.

코랩-깃허브 연동 방식

별도 설정이 아니라 URL 경로 규칙으로 동작합니다

```
colab.research.google.com / github / Song-yiJung / korean-ocr-lectures / ...
```

코랩 도메인

이하가 깃허브 저장소 경로

코랩이 github 이하의 경로를 해석하여 해당 저장소의 노트북을 로드합니다.
사전에 연결을 설정한 것이 아니라, 주소를 이 형식으로 기술하면 그렇게 동작합니다.

여기에서 도출되는 두 가지

· 다운로드 절차가 없습니다

파일을 수신한 것이 아니라 URL로 로드한 것입니다.

· ①칸의 wget도 동일 저장소를 참조합니다

실습 자료도 같은 경로에서 받습니다. 그래서 URL에 폴더명이 그대로 포함됩니다.

구글 코랩 — 클라우드 실행 환경

브라우저는 입출력만 담당하고 연산은 구글 서버에서 수행됩니다

이점

- 설치 과정이 필요 없습니다
- 참가자 전원이 동일 환경에서 실행합니다
- 운영체제와 무관하게 동작합니다
- URL 하나로 접속합니다

제약

- 네트워크 연결이 끊기면 중단됩니다
- 유휴 상태가 지속되면 런타임이 종료됩니다
- 자료를 구글 서버에 업로드해야 합니다
- 무료 사용량에 상한이 있습니다

외부 반출이 불가능한 자료이거나 장시간 연산이 필요한 경우에는 로컬 환경 구축을 검토합니다.

파이썬 — 프로그래밍 언어

설치하여 실행하는 응용 프로그램이 아니라, 명령을 기술하는 언어입니다

3차시에서 사용한 문법 요소

`import fitz` 라이브러리를 불러옵니다

`해상도 = 200` 변수에 값을 할당합니다

`# 뒤의 글` 주석. 실행되지 않습니다

`!pip install` 셸 명령을 실행합니다

`for 쪽 in 쪽목록:` 반복문입니다

`f"p{쪽}.jpg"` 문자열에 값을 삽입합니다

이 여섯 개로 3차시 전 과정이 기술됩니다.
문법 암기는 필요하지 않으며, 각 행이
무엇을 수행하는지 식별하면 충분합니다.

이 워크숍의 목표는 코드 작성 능력이
아니라, 코드의 수행 결과를 판독하고
원본과 대조하는 능력입니다.

인문학 연구에서 파이썬이 사용되는 이유

언어 자체의 특성보다 생태계와 학술 관행에 기인합니다

1 라이브러리 생태계

OCR, 형태소 분석, 지리정보, 연결망 분석 등 인문학 연구에 사용되는 라이브러리 대부분이 파이썬으로 배포됩니다. 기존 도구를 사용하려면 해당 언어를 써야 합니다.

2 낮은 문법 진입 장벽

구문이 자연어에 가깝게 설계되어 있습니다. 3차시 코드에서 for와 if의 기능을 추론할 수 있었다면 그것이 이 언어의 특성입니다.

3 무료 · 플랫폼 독립

사용료가 없고 운영체제와 무관하게 동일한 코드가 실행됩니다. 별도 예산 없이 착수할 수 있고 코드를 그대로 배포할 수 있습니다.

4 학술 관행

논문 부록으로 분석 코드를 공개하는 관행이 정착했습니다. 선행 연구를 검증하거나 확장하려면 코드를 판독할 수 있어야 합니다.

운영체제 독립과 GPU

서로 무관한 사안이므로 구분해 둡니다

운영체제와 무관하다는 것

코드가 내 PC가 아니라 구글의 리눅스 서버에서 실행되기 때문입니다. 내 PC가 윈도우든 맥이든 실행 결과가 동일합니다. 로컬에 설치할 경우에는 운영체제별로 설치 절차와 경로 표기가 달라집니다.

GPU (그래픽 처리 장치)

단순 연산을 대량으로 병렬 처리하는 장치입니다. 딥러닝 모델의 학습과 추론에 주로 사용됩니다. 코랩에서는 「런타임 → 런타임 유형 변경」에서 선택할 수 있습니다.

이 실습에는 GPU가 필요하지 않습니다.

문자 인식과 교정 연산은 구글 비전 · 제미나이 서버에서 수행됩니다. 코랩 런타임은 PDF를 이미지로 변환하고 API를 호출하는 역할만 합니다.

런타임 유형을 GPU로 변경해도 속도는 개선되지 않으며, 무료 사용량만 추가로 소모됩니다.

런타임과 가상 머신

런타임의 실체는 구글 서버에 할당된 가상 머신 한 대입니다

가상 머신 (Virtual Machine)

물리 서버 한 대 위에 소프트웨어로 분리해 만든 독립된 컴퓨터입니다. 각각 자체 운영체제와 파일 시스템을 가지며 서로 간섭하지 않습니다.

- 1 할당** 서버 한 대의 자원을 다수 사용자에게 분할해 배정합니다. 참가자마다 각각 한 대씩 배정됩니다.
- 2 회수** 종료되면 해당 가상 머신이 회수됩니다. 그 안의 파일 시스템도 함께 폐기됩니다.
- 3 재배포** 다시 시작하면 초기 상태의 새 가상 머신이 배정됩니다. 이전 상태는 인계되지 않습니다.

그러므로 「초기화」보다 「새 컴퓨터로 교체」에 가깝습니다. 무료 등급의 가상 머신은 메모리 약 12~13GB 수준입니다.

유휴 상태와 세션 한도

종료 조건을 수치로 정리합니다

유휴 상태의 정의

셀이 실행되고 있지 않고, 노트북 탭에 대한 입력(클릭 · 타이핑 · 스크롤)도 없는 상태입니다. 탭을 열어 두기만 한 것은 활동으로 계산되지 않습니다.

유휴 시간 초과

약 90분

입력이 없는 상태가 이어지면 연결이 끊깁니다

최대 연속 사용

약 12시간

작업 중이어도 이 시점에서 종료됩니다

실습 중 질의응답이 길어지면 유휴 시간이 누적됩니다. 셀을 하나 실행하면 유휴 시간이 초기화됩니다.

위 수치는 무료 등급에서 통상 관측되는 값이며 구글이 보장하는 값은 아닙니다. 전체 이용량에 따라 변동합니다.

런타임 종료

노트북 파일과 런타임은 별개의 대상입니다

노트북 파일 (.ipynb)

코드와 출력이 저장됩니다 — 유지됩니다

런타임

코드가 실행되던 가상 머신 — 초기화됩니다

종료 조건

유틸 상태 지속 · 브라우저 종료 · 최대 연속 사용 시간 초과

종료 후 필요한 조치는 ①칸부터의 재실행뿐입니다.

키는 재발급 대상이 아닙니다. 비전 json은 드라이브에, 제미나이 문자열은 코랩 보안 비밀에 유지됩니다.

런타임 종료 시의 상태

유지되는 항목과 초기화되는 항목

유지

노트북 사본

내 드라이브

비전 키 .json

드라이브 keys 폴더

제미나이 키 문자열

코랩 보안 비밀

⑪칸에서 저장한 결과

내 드라이브 / ocr_결과

초기화

설치한 라이브러리

①칸이 재설치

내려받은 PDF

①칸이 재수신

step0·1·2 폴더

④⑥⑨칸이 재생성

드라이브 마운트

②칸이 재연결

우측 항목은 모두 해당 칸의 재실행으로 복구됩니다. 재발급이나 재업로드가 필요한 항목은 없습니다.

파일의 저장 위치 세 가지

위치별로 지속성과 접근 권한이 다릅니다

원격 저장소

깃허브

읽기 권한만 있습니다

클라우드 저장소

내 구글 드라이브

계정에 영구 보관됩니다

실행 환경

코랩 런타임

종료 시 초기화됩니다

→
사본 저장

→
드라이브 마운트

깃허브의 노트북을 열고 → 내 드라이브에 사본을 생성한 뒤 → 런타임에 드라이브를 마운트하여 사용합니다.

내 PC · 런타임 · 드라이브의 관계

런타임은 내 PC의 파일 시스템에 접근할 수 없습니다

내 PC

파일 탐색기 · 바탕화면

코랩 런타임

구글 서버의 가상 머신

내 구글 드라이브

구글 계정의 저장소

직접 연결 없음

마운트로 연결

내 PC의 파일을 런타임이 읽게 하는 두 가지 경로

① 드라이브에 업로드한 뒤 마운트

드라이브에 남으므로 런타임이 종료되어도 유지됩니다. 키 파일은 이 방식을 씁니다.

② 코랩 파일 창에 직접 업로드

런타임 내부에만 존재하므로 종료 시 함께 사라집니다. 일회성 자료에만 씁니다.

「키 파일이 드라이브에 있어야 한다」는 것은 드라이브에 사본이 있어야 한다는 뜻이며, 내 PC에서 삭제하라는 뜻이 아닙니다.

드라이브 마운트

외부 저장소를 파일 시스템의 특정 경로에 대응시키는 조작입니다

마운트란 별도의 저장 장치를 운영체제의 디렉터리 하나에 연결하여, 그 아래의 내용을 일반 파일처럼 읽고 쓸 수 있게 만드는 것입니다.

```
drive.mount("/content/drive")
```

실행 후의 경로 대응

내 드라이브 최상위 `/content/drive/MyDrive`

keys 폴더 `/content/drive/MyDrive/keys`

1 복사가 아니라 연결입니다

런타임에 파일이 복제되지 않습니다. 마운트된 경로를 통해 드라이브의 실제 파일을 직접 읽고 씁니다.

2 쓰기도 드라이브에 반영됩니다

마운트된 경로에 파일을 저장하면 드라이브에 기록되며, 런타임이 종료되어도 남습니다. ⑪ 칸이 이 성질을 이용합니다.

3 런타임마다 다시 해야 합니다

마운트는 런타임에 설정되는 것이므로, 새 가상 머신이 배정되면 다시 실행해야 합니다.

사본 저장이 필요한 이유

깃허브에서 로드한 노트북에는 쓰기 권한이 없습니다

사본을 생성하지 않은 경우

수정 내용이 원본에 반영되지 않습니다.
쓰기 권한이 없기 때문입니다.
브라우저 종료 시 수정 내용이 소실됩니다.

사본을 생성한 경우

노트북이 내 드라이브로 복사됩니다.
이후 수정 내용이 저장됩니다.
내 드라이브 / Colab Notebooks 에 생성됩니다.

파일 → 드라이브에 사본 저장 · 제목이 「...의 사본」으로 변경되면 완료입니다

API — 프로그램 간 호출 규격

제미나이는 웹에서도 사용 가능한데 키 발급이 필요한 이유

웹 인터페이스

사람이 직접 입력하고 화면으로 확인합니다.
단건 처리에는 이 방식으로 충분합니다.

API

프로그램이 규정된 형식으로 요청하고
규정된 형식으로 응답을 수신합니다.

요청과 응답의 형식이 고정되어 있습니다

형식이 고정되어야 프로그램이 결과를 자동으로 처리할 수 있습니다.

⑥칸의 `document_text_detection( image=..., image_context=... )` 이 요청이고 괄호 안이 매개변수이며, 응답도 규정된 구조로 반환됩니다.

두 API의 기능 비교

기능이 상보적이므로 두 단계로 연결합니다

구글 비전

제미나이

입력

이미지

이미지와 텍스트

기능

문자 형태를 인식합니다

문맥을 반영해 교정합니다

판단 근거

획의 형태

언어 모델상의 확률

비용 · 속도

저비용 · 고속

고비용 · 저속

한계

문맥 오류를 검출하지 못합니다

문맥에 맞는 내용을 생성할 수 있습니다

Holley(2008)가 제안했으나 당시 구현하지 못한 「문자 형태 판별 + 언어 모델」 결합이 ⑥칸과 ⑨칸의 구성에 해당합니다.

API 키 — 호출자 식별 자격증명

호출이 유료이므로 계정별로 사용량을 집계합니다

필요한 이유

사용량을 계정별로 집계해
과금하기 위해서입니다.

노출 금지 이유

유출되면 타인의 호출이
본인 계정에 과금됩니다.

과금 구간

①~⑤칸은 런타임 내부
처리이므로 무료입니다.
⑥칸부터 외부 호출입니다.

비전은 월 1,000장까지 무료입니다. 제미나이는 페이지당 수 원 수준이며, 본 실습 분량은 십 원 단위입니다.

비용은 제약 요인이 아닙니다. 다만 키를 타인에게 전달하지 않도록 합니다.

두 키의 형식과 보관 위치

혼동이 가장 빈번한 지점입니다

구글 비전

형식	.json 파일
보관 위치	드라이브 keys 폴더
코드상 저장 대상	환경변수
⑥칸에서	키 이름이 나타나지 않습니다

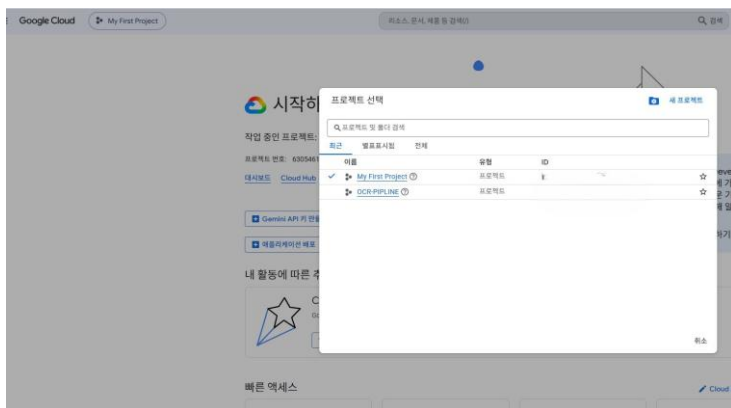
제미나이

형식	문자열
보관 위치	코랩 보안 비밀
코드상 저장 대상	GEMINI_KEY 변수
⑨칸에서	인자로 직접 전달합니다

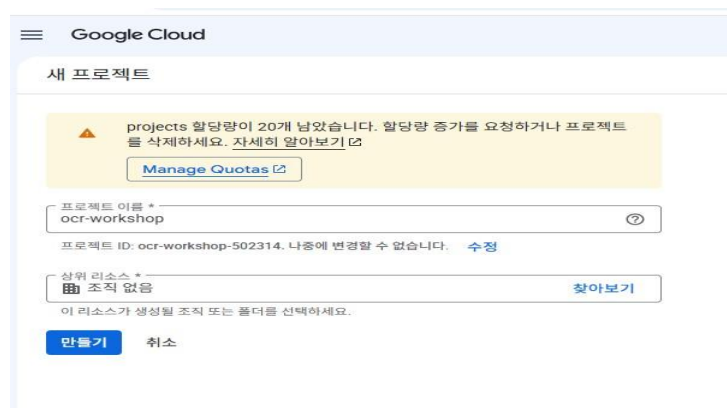
저장 대상이 다르므로 후속 칸에서의 표현 방식도 다릅니다. 오류가 아닙니다.

비전 키 ① 프로젝트 생성과 결제 계정 등록

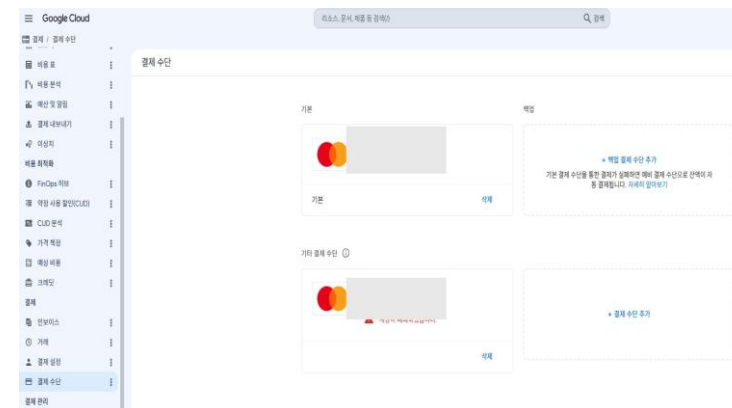
반드시 개인 구글 계정으로 진행합니다 · 기관 계정은 관리자 정책으로 막혀 있는 경우가 많습니다



프로젝트 선택 팝업 → [새 프로젝트]



이름은 ocr-workshop · 위치는 조직 없음



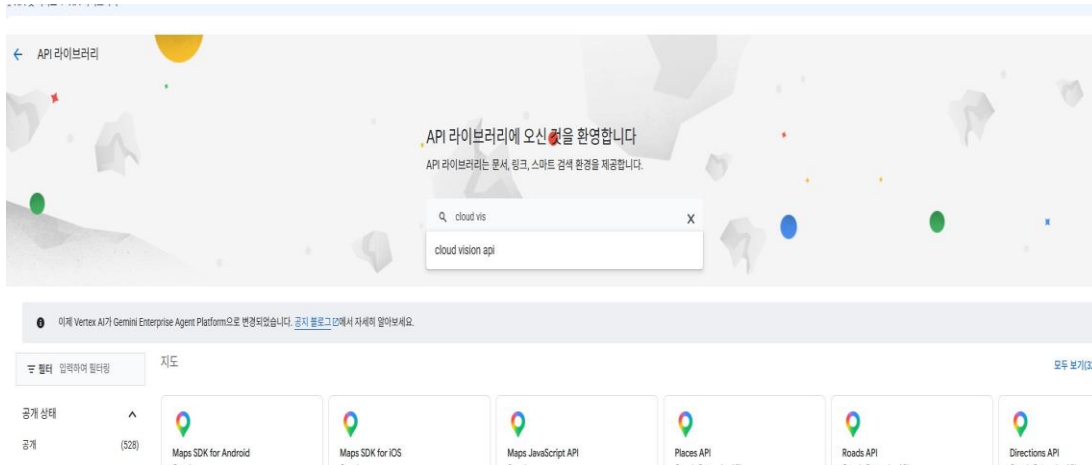
[결제] → [결제 수단]에 카드가 「기본」으로

1. console.cloud.google.com 에 접속합니다. 오른쪽 위 프로필 아이콘으로 개인 계정인지 확인합니다.
2. 왼쪽 위 프로젝트 선택 버튼 → 팝업 오른쪽 위 [새 프로젝트] → 이름 ocr-workshop (영문 소문자 · 숫자 · 하이픈만).
3. 만든 뒤 다시 프로젝트 선택 버튼을 눌러 ocr-workshop 으로 전환합니다. 이후 모든 작업은 이 상태에서 합니다.
4. 탐색 메뉴(≡) → [결제] → [결제 계정 만들기] → 계정 유형 「개인」 → 카드 정보 입력.
5. 등록만으로는 청구되지 않습니다. 약 1달러의 승인 문자는 본인 확인용이며 자동으로 취소됩니다.

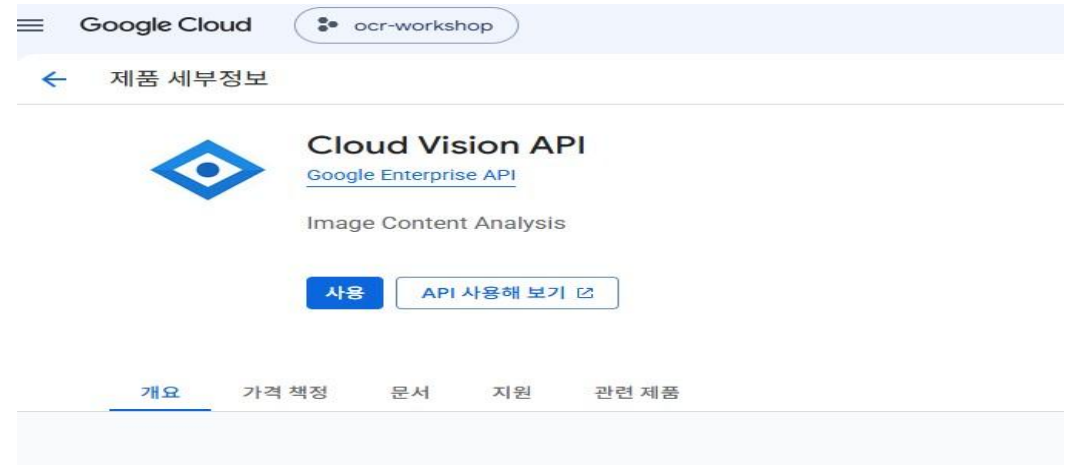
확인 지점 — 탐색 메뉴(≡) → [결제] → [계정 관리]의 연결된 프로젝트 목록에 ocr-workshop 이 보이면 완료입니다.

비전 키 ② Cloud Vision API 켜기

구글 클라우드의 기능은 기본적으로 모두 꺼져 있습니다



[API 및 서비스] → [라이브러리] 검색창에 Cloud Vision API

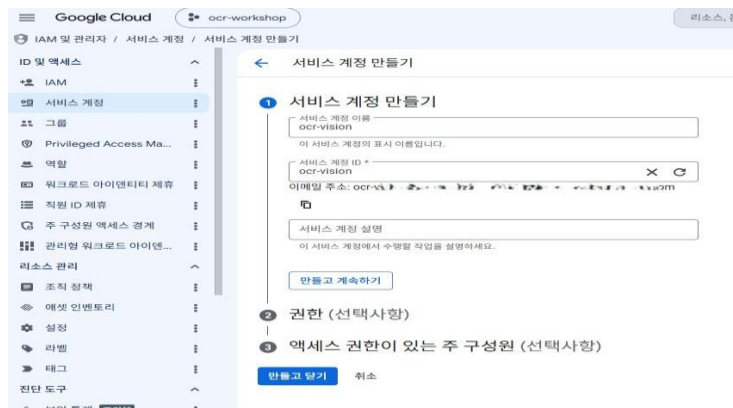


제품 세부정보 화면의 파란색 [사용] 버튼

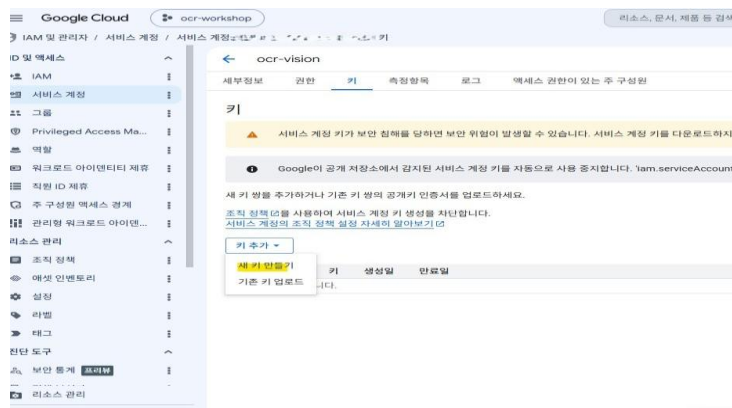
1. 탐색 메뉴(☰) → [API 및 서비스]에 마우스를 올리면 옆에 [라이브러리]가 나타납니다.
2. 검색창에 Cloud Vision API 를 입력하고, 이름이 정확히 일치하는 카드를 클릭합니다.
3. 파란색 [사용] 버튼을 클릭합니다.
4. 위쪽 버튼이 [API 사용중지]로 바뀌면 켜진 것입니다. 처음부터 [관리]나 [API 사용중지]로 보이면 이미 켜진 상태입니다.

비전 키 ③ 서비스 계정 생성과 JSON 키 내려받기

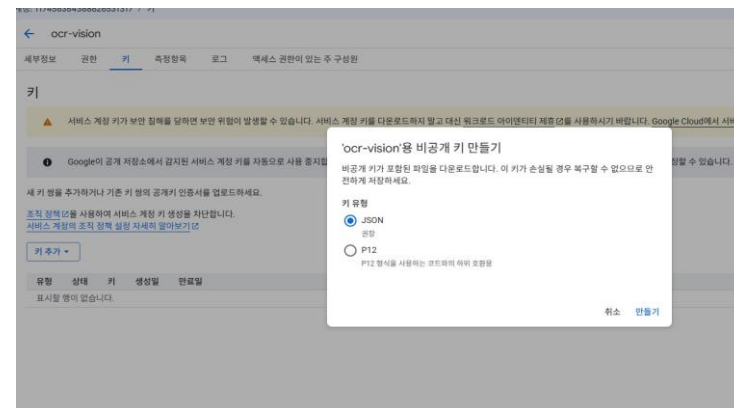
서비스 계정은 사람이 아닌 프로그램에 부여하는 계정이고, JSON 파일이 그 자격증명입니다



이름은 ocr-vision · 권한 단계는 건너뜁니다



[키] 탭 → [키 추가] → [새 키 만들기]



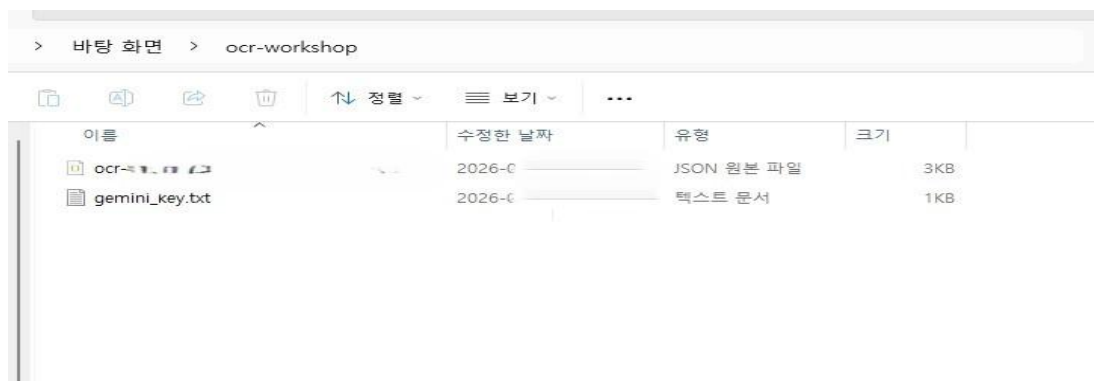
키 유형 JSON 확인 후 [만들기]

1. 탐색 메뉴(☰) → [IAM 및 관리자] → [서비스 계정] → [+ 서비스 계정 만들기].
2. 이름에 ocr-vision 을 입력합니다. 서비스 계정 ID는 자동으로 채워지므로 건드리지 않습니다.
3. 「권한」과 「액세스 권한이 있는 주 구성원」은 모두 선택사항이므로 입력하지 않고 [만들고 닫기]를 누릅니다.
4. 목록에서 만든 계정의 이메일 주소를 클릭 → 위쪽 [키] 탭 → [키 추가] → [새 키 만들기] → JSON → [만들기].
5. 파일이 자동으로 내려받아집니다. 바탕화면에 ocr-workshop 폴더를 만들어 옮겨 둡니다.

이 파일은 비밀번호와 같습니다. 이메일 · 메신저 · 게시판에 올리지 않습니다. 분실하면 4번을 반복해 새 키를 받으면 됩니다.

비전 키 ④ JSON 파일을 드라이브 keys 폴더에 올리기

이 단계를 건너뛰면 실습을 진행할 수 없습니다



바탕화면 ocr-workshop 폴더 — JSON 파일과 gemini_key.txt

화면 캡처 자리

구글 드라이브의 내 드라이브 > keys 폴더와
그 안의 .json 파일이 함께 보이도록
(문서에 이 화면의 캡처가 없습니다)

1. drive.google.com 에 접속합니다. 지금까지 쓴 것과 같은 개인 계정인지 오른쪽 위 프로필로 확인합니다.
2. [+ 신규] → [새 폴더] → 이름에 keys 를 입력하고 만듭니다. 소문자여야 합니다.
3. 만든 keys 폴더를 열고, 바탕화면 ocr-workshop 폴더의 .json 파일을 끌어다 놓습니다.
4. gemini_key.txt 는 올리지 않습니다. 문자열 키는 파일이 아니라 코랩 보안 비밀에 등록합니다.

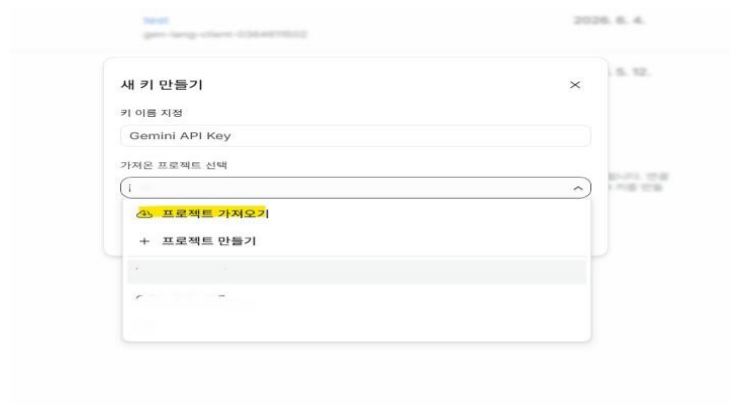
런타임은 사용자 PC의 파일에 접근할 수 없습니다. 바탕화면에만 두면 실습 중 「키 파일을 찾지 못했습니다」가 뜹니다.

제미나이 키 발급 — 구글 AI 스튜디오

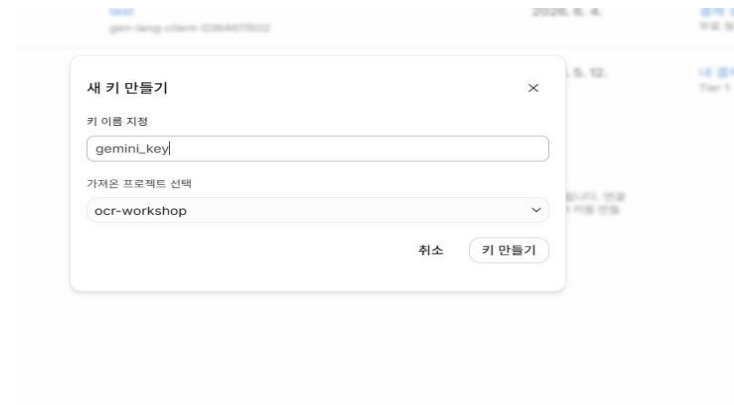
비전 키와 같은 ocr-workshop 프로젝트 안에 만듭니다



왼쪽 [API 키] → 오른쪽 위 [API 키 만들기]



이름 gemini_key · 프로젝트 선택



ocr-workshop 확인 후 [키 만들기]

1. aistudio.google.com 에 접속합니다. 비전 키를 만들 때 쓴 것과 같은 개인 계정으로 로그인합니다.
2. 왼쪽 메뉴 [API 키] → 오른쪽 위 [API 키 만들기].
3. 키 이름에 gemini_key 를 입력합니다.
4. 「가져온 프로젝트 선택」에서 ocr-workshop 을 고릅니다. 목록에 없으면 [프로젝트 가져오기]로 먼저 가져옵니다.
5. [키 만들기]를 누릅니다. 소요 시간은 약 5분입니다.

프로젝트를 반드시 ocr-workshop 으로 지정합니다. 다른 프로젝트에 만들면 뒤 슬라이드의 등급 조건이 달라집니다.

제미나이 키 등록 — 코랩 보안 비밀

복사한 문자열을 노트북이 읽을 수 있는 자리에 넣습니다

보안 비밀

+ 새 보안 비밀 추가

노트북 액세스

이름

값

1



2

GEMINI_API_KEY

3

.....

```
from google.colab import userdata
userdata.get('GEMINI_API_KEY')
```

등록하면 이 코드로 읽습니다 — ②칸이 하는 일입니다

1

노트북 액세스

이 토글이 꺼져 있으면 저장은 되어도 노트북이 값을 읽지 못합니다

2

이름

GEMINI_API_KEY — 대소문자와 밑줄까지 일치해야 합니다

3

값

발급받은 문자열을 붙여넣습니다. 저장 후에는 다시 표시되지 않습니다



코랩 보안 비밀 창
토글이 켜진 상태

키 세부정보 창의 복사 아이콘

두 키의 발급 조건 · 등급 · 비용

두 키를 같은 ocr-workshop 프로젝트 안에 만든 데에는 이유가 있습니다

구글 비전

발급처	구글 클라우드 콘솔
결과물	.json 파일
소요 시간	약 25분
프로젝트	ocr-workshop
등급	결제 계정 연결됨
비용	월 1,000장까지 무료

제미나이

구글 AI 스튜디오
문자열
약 5분
ocr-workshop (동일)
유료 등급으로 동작
이미지 한 장에 몇 원 수준

제미나이를 유료 등급으로 쓰는 이유

- 무료 등급에는 분당 · 일일 호출 한도가 있어 여러 사람이 동시에 실습하면 중간에 막힙니다.
- 무료 등급에서는 입력한 내용이 구글의 모델 개선에 사용됩니다. 미간행 소장 자료를 다루는 연구에서는 소장기관과의 이용 조건에 걸릴 수 있습니다.

확인 방법

AI 스튜디오의 API 키 페이지에서 해당 프로젝트의 Plan 칸이 「Paid」로 표시되면 맞습니다.

키 로드 — 저장된 키를 코드가 참조할 수 있게 만드는 절차

키를 새로 만들거나 옮기는 과정이 아닙니다

구글 비전 — 간접 참조

- ① 드라이브 keys 폴더에서 .json 파일의 경로를 찾습니다
- ② 그 경로를 환경변수 `GOOGLE_APPLICATION_CREDENTIALS`에 등록합니다
- ③ `vision.ImageAnnotatorClient()`를 생성하면 라이브러리가 이 변수를 읽고 파일을 열어 인증 정보를 구성합니다

코드는 키의 내용을 직접 다루지 않고 위치만 지정합니다

제미나이 — 직접 전달

- ① `userdata.get()`이 코랩 보안 비밀에서 문자열을 읽습니다
- ② 읽은 값을 `GEMINI_KEY` 변수에 담습니다
- ③ `genai.Client(api_key=GEMINI_KEY)`에 인자로 전달하면 이후 요청마다 이 문자열이 함께 전송됩니다

코드가 키 문자열 자체를 변수로 보유합니다

「로드」는 저장된 값을 읽어 프로그램이 참조할 수 있는 상태로 만드는 것을 가리킵니다.

노트북의 구성 — ①~⑫칸

직접 수정하는 칸은 ③ 하나이고, 나머지는 실행만 합니다

① 설치와 자료 받기 실행만

② 키 로드 실행 + 권한 허용

③ 설정 직접 수정

④ PDF를 이미지로 변환 실행만

⑤ 생성된 이미지 확인 실행만

⑥ 구글 비전 호출 실행만 · 과금 시작

⑦ 비전 결과 확인 실행만

⑧ 비전이 알려 준 위치 보기 실행만

⑨ 제미나이 교정 실행만

⑩ 비전과 교정 대조 실행만

⑪ 드라이브에 저장 실행만

⑫ 내 컴퓨터로 내려받기 실행만

앞 칸의 출력을 뒤 칸이 받아 쓰므로 위에서부터 차례로 실행합니다. ②칸은 앞에서 따로 다루었습니다.

②칸의 구성 — 세 단계

단일 셀이지만 수행 작업은 세 가지입니다

1

드라이브 마운트

권한 요청 창이 표시됩니다. 계정을 선택하고 허용합니다.

2

비전 키 로드

마운트된 드라이브의 keys 폴더에서 .json 파일을 탐색합니다.

3

제미나이 키 로드

코랩 보안 비밀에 등록된 문자열을 읽습니다.

실행 후 두 행이 출력됩니다. 두 행 모두 「확인」이어야 이후 단계가 진행됩니다.

① 드라이브 마운트

코랩 런타임은 사용자 PC의 파일에 접근할 수 없습니다

```
try:
    drive.mount("/content/drive")
    연결됨 = os.path.isdir("/content/drive/MyDrive")
except Exception:
    연결됨 = False
```

권한 요청 창

계정을 선택하고 허용합니다.
런타임이 교체될 때마다 반복합니다.

마운트가 필요한 지점은 두 곳입니다

비전 키 로드 시 — 키 파일의 사본이 드라이브에 있어야 런타임이 읽을 수 있습니다.

⑪칸의 결과 저장 시 — 런타임은 초기화되지만 드라이브의 내용은 유지됩니다.

실제 화면: 이 셀 실행 시 표시되는 권한 요청 창을 캡처해 삽입합니다.

② 비전 키 로드

파일명이 계정마다 다르므로 와일드카드로 탐색합니다

```
열쇠들 = glob.glob("/content/drive/MyDrive/keys/*.json") if 연결됨 else []

if 열쇠들:
    os.environ["GOOGLE_APPLICATION_CREDENTIALS"] = 열쇠들[0]
    비전상태 = f"확인   파일 {os.path.basename(열쇠들[0])}"
elif not 연결됨:
    비전상태 = "없음   드라이브가 연결되지 않았습니다"
elif not os.path.isdir("/content/drive/MyDrive/keys"):
    비전상태 = "없음   keys 폴더가 안 보입니다 - 연결한 계정이 다를 수 있습니다"
else:
    비전상태 = "없음   keys 폴더에 .json 이 없습니다"
```

*

임의의 문자열과 일치하는 와일드카드입니다

`os.environ`

키 파일 경로를 환경변수에 등록합니다. 비전 라이브러리가 이 변수를 참조합니다

elif 의 분기

실패 원인이 셋이므로 각각을 구분하여 출력합니다

③ 제미나이 키 로드

등록명이 한 글자라도 다르면 조회되지 않습니다

```
try:
    GEMINI_KEY = userdata.get("GEMINI_API_KEY")
except Exception:
    GEMINI_KEY = ""

if not GEMINI_KEY:
    from getpass import getpass
    GEMINI_KEY = getpass("보안 비밀번호에서 조회되지 않았습니다. 키를 입력하십시오: ").strip()
```

등록명

GEMINI_API_KEY

대소문자와 밑줄까지 일치해야 합니다

노트북 액세스 토글

비활성 상태에서는 저장은 되지만 노트북이 값을 읽지 못합니다.

3차시 오류의 다수가 이 항목이었습니다.

실제 화면: 코랩 보안 비밀번호 창에서 등록명과 토글이 함께 보이도록 캡처해 삽입합니다.

②칸의 정상 실행 출력

실행 후 두 행이 출력됩니다

```
구글 비전   확인   파일 my-project-3f2a1c.json
제미나이   확인   문자열 39자
```

두 행이 함께 출력되는 것이 확인 기준입니다. 키가 두 개라는 사실이 출력으로 검증됩니다.

한 행만 「확인」인 경우

비전만 확인된 경우 ⑨칸에서, 제미나이만 확인된 경우 ⑥칸에서 실행이 중단됩니다.
이 단계에서 확인하지 않으면 실습 도중에는 개별 대응이 어렵습니다.

「없음」 출력 시 원인과 조치

네 가지 중 하나에 해당합니다

드라이브가 연결되지 않았습니다

권한 요청 창에서 취소한 경우입니다. ②칸을 재실행합니다.

keys 폴더가 조회되지 않습니다

마운트한 계정이 다를 가능성이 높습니다. 개인 계정에 폴더를 생성하고 기관 계정으로 마운트한 경우 이 상태가 됩니다.

keys 폴더에 .json 이 없습니다

키 파일이 사용자 PC에만 존재합니다. 드라이브의 keys 폴더로 업로드합니다.

제미나이 — 등록명 또는 토글

등록명이 일치하지 않거나 노트북 액세스 토글이 비활성 상태입니다.

네 경우 모두 키 재발급은 불필요합니다. 파일 위치를 옮기거나 등록명을 수정하면 해결됩니다.

①칸 — 설치와 자료 받기

런타임이 새로 지정될 때마다 다시 실행합니다



```
!pip install -q google-cloud-vision google-genai pymupdf pillow
```

```
저장소 = "https://raw.githubusercontent.com/Song-yiJung/korean-ocr-lectures/main"
```

```
!wget -q "${저장소}/2026-08-pnu-workshop/session3/data/drag/drag_A_busan60.pdf" -O "부산광복60년_201-205.pdf"
```

출력 부산광복60년_201-205.pdf 2,417KB — 준비 끝

`!pip install`

라이브러리를 설치합니다. 가상 머신이 초기 상태이므로 매번 필요합니다

`!wget`

깃허브 저장소에서 실습 자료를 내려받습니다. 4번 슬라이드에서 본 그 저장소입니다

크기 확인

받은 파일이 100KB를 넘는지 검사해 「준비 끝」을 출력합니다. 이 문구가 안 보이면 실패한 것입니다

③칸 — 설정 · 직접 수정하는 유일한 칸

다섯 개의 변수와 하나의 지시문으로 구성됩니다



```
PDF = glob.glob("**광복*.pdf")[0]      # 읽을 PDF

발췌시작 = 201                          # 이 파일의 첫 쪽이 원본 몇 쪽인지
쪽목록 = [201]                          # ← 읽을 쪽 (원본 쪽 번호로 적습니다)

해상도 = 200                            # dpi

교정지시 = """아래는 한국 현대 인쇄 자료를 OCR 로 읽은 결과다.
1. 원문에 없는 내용을 절대 추가하지 마라.
2. 판독이 불확실한 글자는 추측하지 말고 □ 로 표시하라. ..."""
```

출력 자료 : 부산광복60년_201-205.pdf
 읽을 쪽: [201] (1장)

교정지시가 제미나이의 동작을 결정합니다

③칸은 이 문장을 그대로 받아 제미나이에 전달합니다. 「없는 내용을 추가하지 마라」와 「불확실하면 □로 표시하라」가 생성 억제 장치입니다. 자료 성격이 바뀌면 이 문장을 고칩니다.

④·⑤칸 — PDF를 이미지로 변환하고 확인

구글 비전은 PDF가 아니라 이미지를 입력으로 받습니다

④칸



```
문서 = fitz.open(PDF)
사진들 = []
for 쪽 in 쪽목록:
    이름 = f"step0_사진/p{쪽}.jpg"
    문서[쪽 - 발췌시작].get_pixmap(dpi=해상도).save(이름)
```

출력 p201 → step0_사진/p201.jpg

사진 1장 준비 완료

⑤칸



```
그림 = Image.open(사진들[0][1])
그림.resize((그림.size[0] // 2, 그림.size[1] // 2))
```

출력 step0_사진/p201.jpg 1654 x 2339 (화면에는 절반 크기의 이미지가 표시됩니다)

④칸의 사진들 목록은 (쪽 번호, 파일 경로) 쌍으로 구성되며 ⑥칸과 ⑨칸이 이를 그대로 받아 씁니다.

⑥·⑦칸 — 구글 비전 호출과 결과 확인

여기서부터 외부 호출이므로 과금 구간입니다

⑥칸



```
비전 = vision.ImageAnnotatorClient()
설정 = vision.ImageContext(language_hints=["ko", "zh", "en"])
비전응답 = {} # ⑧칸에서 좌표를 꺼내 쓰려고 남겨 둡니다

for 쪽, 사진 in 사진들:
    응답 = 비전.document_text_detection(image=vision.Image(content=open(사진, "rb").read()),
                                         image_context=설정)
```

출력 p201 1,842자 자신 없어 한 낱말 7개

모두 1,842자

⑦칸



```
print(비전결과[쪽목록[0]][:400])
```

출력 (인식된 본문의 앞 400자가 그대로 출력됩니다)

「자신 없어 한 낱말」은 비전이 반환한 확신도가 0.80 미만인 낱말의 수입입니다. 어디가 틀렸는지가 아니라 몇 개가 불안한지만 알려 줍니다.

⑧칸 — 응답의 구조와 좌표

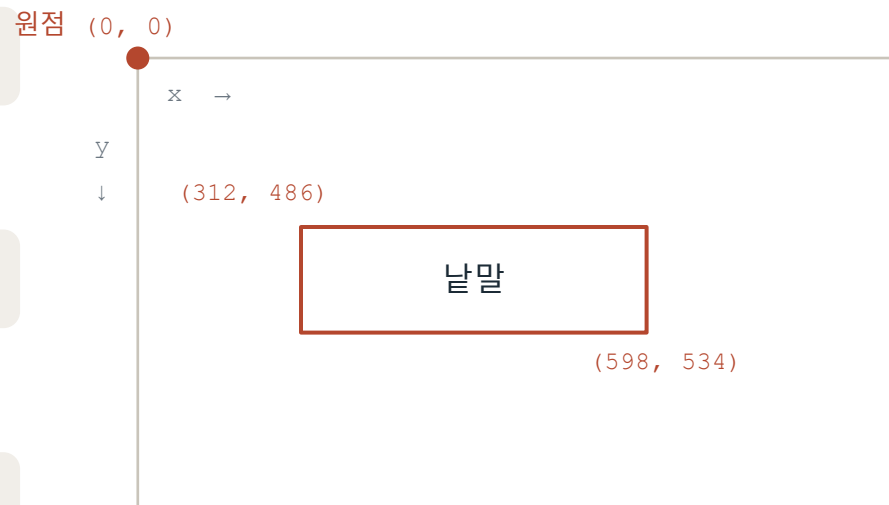
비전은 글자와 함께 그 글자가 지면의 어디에 있는지도 돌려줍니다

응답은 다섯 겹으로 중첩되어 있습니다

<code>pages</code>	면	폭 · 높이, 사각형 좌표
<code>blocks</code>	지면의 구획	사각형 좌표, 구획 종류
<code>paragraphs</code>	문단	사각형 좌표
<code>words</code>	낱말	사각형 좌표, 확신도
<code>symbols</code>	낱글자	사각형 좌표

⑥칸에서 확신도를 셀 때 연 for 네 겹이 이 구조입니다. 같은 자리에서 좌표를 꺼냅니다.

`bounding_box.vertices` — 사각형의 꼭짓점 넷



원점은 이미지의 왼쪽 위입니다. x 는 오른쪽으로, y 는 아래로 커집니다. 단위는 픽셀입니다.

`full_text_annotation.text` 는 이 구조를 비전이 읽기 순서라고 판단한 대로 이어 붙인 결과입니다. 구조가 원본이고, 텍스트는 파생물입니다.

⑧칸 실행 화면 — 좌표를 원본 위에 그려 봅니다

확신도가 몇 개인지가 아니라 어디에 있는지가 드러납니다



```
def 사각형(요소, 색, 굵기):  
    점 = [(꼭짓점.x, 꼭짓점.y) for 꼭짓점 in 요소.bounding_box.vertices]  
    그리기.line(점 + [점[0]], fill=색, width=굵기)  
  
for 면 in 비전응답[쪽].full_text_annotation.pages:  
    for 덩어리 in 면.blocks:  
        사각형(덩어리, (30, 90, 200), 6)
```

출력 첫 낱말 : 여성사회의
 그 네 꼭짓점 : [(312, 486), (598, 486), (598, 534), (312, 534)]

파란 테두리 = 덩어리 초록 = 낱말 빨강 = 확신도 0.80 미만
낱말 412개 중 빨강 7개

화면 읽는 법

파란 테두리는 지면의 구획, 초록은 낱말입니다.
빨간 낱말이 ⑥칸에서 「자신 없어 한 낱말」로
개수만 세어졌던 그것입니다.

좌표가 연구에서 하는 일

다단 지면은 x의 무리로 단을 나눠 재배열하고,
표와 명단은 행을 y, 열을 x의 무리로 복원합니다.
확신도가 낮은 낱말은 원본의 그 지점을 바로 찾습니다.

제미나이 응답에는 좌표가 없습니다. 위치를 쓰려면 ⑥칸의 응답을 남겨 이 칸에서 꺼내야 합니다.

⑨칸 — 제미나이 교정

이미지와 비전 결과를 함께 전달합니다



```
제미나이 = genai.Client(api_key=GEMINI_KEY)
```

```
응답 = 제미나이.models.generate_content(  
    model="gemini-3.5-flash",  
    contents=[ types.Part.from_bytes(data=open(사진, "rb").read(), mime_type="image/jpeg"),  
               교정지시 + 비전결과[쪽] ],  
    config=types.GenerateContentConfig(temperature=0.0, max_output_tokens=32768))
```

```
글 = 응답.text or ""          # 답이 비어 올 때가 있어 빈 글로 받아 둡니다
```

출력 p201 1,842자 → 1,905자 (+63)

교정 끝

`contents = [ 이미지 , 텍스트 ]`

제미나이는 두 가지를 함께 받습니다. 비전은 이미지만 받았습니다

`temperature = 0.0`

생성의 무작위성을 최소화합니다

`응답.text or ""`

응답이 비어 반환되는 경우가 있어 빈 문자열로 받습니다. 이 처리가 없으면 저장 단계에서 오류가 납니다

⑩칸 — 비전 결과와 교정 결과의 대조

무엇이 바뀌었는지를 줄 단위로 표시합니다



```
바뀐것 = [줄 for 줄 in difflib.unified_diff(앞, 뒤, lineterm="", n=0)  
          if 줄[:3] not in ("---", "+++", "@@ ")]
```

출력

```
== p201 ==  
-여성사회의 변화  
+여성사회의 변화  
-1 9 4 5년  
+1945년
```

(화면 예시입니다. 실제 출력은 자료에 따라 달라집니다)

읽는 법

-로 시작하는 줄이 비전 결과,
+로 시작하는 줄이 교정 결과입니다.

이 칸이 검증의 출발점입니다

바뀐 곳이 곧 제미나이가 개입한 곳입니다.
원본과 대조할 때 이 줄들을 먼저 봅니다.

⑪·⑫칸 — 드라이브에 저장하고 내려받기

두 곳에 남겨 두면 런타임이 종료되어도 잃지 않습니다

⑪칸 드라이브에 저장



```
저장자리 = "/content/drive/MyDrive/ocr_결과"  
for 폴더 in ("step0_사진", "step1_비전", "step2_교정"):  
    shutil.copytree(폴더, f"{저장자리}/{폴더}", dirs_exist_ok=True)
```

출력 step0_사진/p201.jpg
 step1_비전/p201.txt
 step2_교정/p201.txt

내 드라이브 / ocr_결과 파일 3개 — 창을 닫아도 남습니다

⑫칸 내 컴퓨터로 내려받기



```
shutil.make_archive("ocr_결과", "zip", "결과")  
files.download("ocr_결과.zip")
```

출력 ocr_결과.zip 1,102KB

⑫칸이 결과 세 폴더만 따로 모으는 것은, 드라이브가 통째로 마운트되어 있어 전체를 묶으면 키 파일까지 딸려 가기 때문입니다.

파이프라인 구조 — 칸 번호와 데이터 흐름

앞 칸의 출력이 뒤 칸의 입력이 됩니다



④를 건너뛰고 ⑥을 실행하면 정의되지 않은 이름에 대한 오류가 발생합니다. 결함이 아니라 입력 부재입니다.

파이프라인은 특정 도구의 명칭이 아니라 단계의 연결 구조를 가리킵니다. 입력과 출력의 형식이 유지되면 개별 도구는 교체할 수 있습니다.

단계 분리의 목적

중간 산출물이 파일로 보존되기 때문입니다

step0_사진 · step1_비전 · step2_교정 — 세 폴더가 각 단계의 산출물입니다.

단일 처리로 통합하면 오류의 발생 지점을 특정할 수 없습니다.

이미지 해상도, 비전의 인식, 제미나이의 교정 중 어느 단계의 문제인지 구분됩니다.

단계가 분리되어 있으므로 역추적이 가능합니다.

검증 가능한 구조를 확보하는 것이 단계 분리의 목적입니다.